Unit 3

Q. Explain the concept of inheritance in C++ and illustrate how public inheritance affects base class members with an example.

- Inheritance is a key feature of Object-Oriented Programming (OOP) that enables hierarchical classification.
- A general class is created to define traits common to a group of related items.
- This general class is inherited by more specific classes which add unique features.
- The class being inherited is called the base class.
- The class that inherits is called the derived class.
- A derived class can also serve as a base class, enabling multiple levels of inheritance.
- When a class inherits another, the members of the base class become members of the derived class.
- When the base class is inherited by using the private access specifier, all public and protected members of the base class become private members of the derived class.

<u>The general syntax of inheritance is:</u>

```
class derived-class-name : access base-class-name {
  // body of class
};
```

- The `access` specifier can be:
  - public
  - private
  - protected
- If no specifier is given:
  - Default is private if it's a class.
  - Default is public if it's a struct.

<u>Effect of Public Inheritance:</u>

- Public members of the base class become public in the derived class.
- Protected members remain protected in the derived class.
- Private members remain private to the base class and are not accessible in the derived class.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
  int i, j;
public:
  void set(int a, int b) { i = a; j = b; }
  void show() { cout << i << " " << j << "\n"; }
```

```
};

class derived : public base {
  int k;
public:
  derived(int x) { k = x; }
  void showk() { cout << k << "\n"; }
};

int main() {
  derived ob(3);
  ob.set(1, 2);   // access base class member
  ob.show();      // access base class member
  ob.showk();     // access derived class member
  return 0;
}
OUTPUT:
1 2
3
```

Q. Explain the purpose and behavior of protected members in C++ inheritance. Illustrate with an example and give the input and output.

- A protected member in a class:
  - Is not accessible by nonmember functions or objects.
  - Is accessible only by:
    - Members of the same class.
    - Derived classes, if the base is inherited as public or protected.
- Protected provides a way to make members act like private data for the rest of the program, while still being inheritable.
- If the base class is inherited as public, protected members remain protected in the derived class and can be accessed from it.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
protected:
  int i, j;  // private to rest of program, accessible by
derived
public:
  void set(int a, int b) { i = a; j = b; }
  void show() { cout << i << " " << j << "\n"; }
};
```

```
class derived : public base {
  int k;
public:
  // derived class can access i and j directly
  void setk() { k = i * j; }
  void showk() { cout << k << "\n"; }
};

int main() {
  derived ob;
  ob.set(2, 3);     // sets i = 2, j = 3
  ob.show();        // prints i and j
  ob.setk();        // calculates k = i * j = 6
  ob.showk();       // prints k
  return 0;
}
```

Input:

- `ob.set(2, 3);` → sets `i = 2, j = 3`
- `ob.setk();` → calculates `k = i * j = 6`

Output:

```
2 3
6
```

Q. Explain the behavior of protected members in inheritance. How are they accessed in multilevel inheritance? Support your answer with an example and give the output.

- Protected members of a class are:
  - Inaccessible to nonmember functions.
  - Accessible within the class itself and its derived classes.
- When a base class is inherited as public, its protected members:
  - Remain protected in the derived class.
  - Are accessible by that derived class.
- This access continues in multilevel inheritance:
  - If the first derived class inherits the base as public, and the second derived class also inherits the first as public, protected members from the base remain accessible in the second derived class.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
protected:
```

```
  int i, j;
public:
  void set(int a, int b) { i = a; j = b; }
  void show() { cout << i << " " << j << "\n"; }
};

// i and j inherited as protected
class derived1 : public base {
  int k;
public:
  void setk() { k = i * j; }
  void showk() { cout << k << "\n"; }
};

// i and j inherited indirectly through derived1
class derived2 : public derived1 {
  int m;
public:
  void setm() { m = i - j; }
  void showm() { cout << m << "\n"; }
};

int main() {
  derived1 ob1;
  derived2 ob2;

  ob1.set(2, 3);
  ob1.show();     // prints i, j = 2 3
  ob1.setk();
  ob1.showk();    // prints k = 2*3 = 6

  ob2.set(3, 4);
  ob2.show();     // prints i, j = 3 4
  ob2.setk();
  ob2.setm();
  ob2.showk();    // prints k = 3*4 = 12
  ob2.showm();    // prints m = 3-4 = -1

  return 0;
}
```

OUTPUT:

```
2 3
6
3 4
12
-1
```

Q. Explain how multiple base classes are inherited in C++. Illustrate with an example and write the output.

- In C++, a derived class can inherit from multiple base classes.
- To inherit multiple base classes:
  - Use a comma-separated list of base classes in the derived class declaration.
  - Specify an access specifier (e.g., `public`) for each base class.
- All accessible members from both base classes are available in the derived class.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base1 {
protected:
  int x;
public:
  void showx() { cout << x << "\n"; }
};

class base2 {
protected:
  int y;
public:
  void showy() { cout << y << "\n"; }
};

// Inherit from base1 and base2
class derived : public base1, public base2 {
public:
  void set(int i, int j) { x = i; y = j; }
};

int main() {
  derived ob;
  ob.set(10, 20);   // sets x = 10, y = 20
  ob.showx();       // calls base1's function
  ob.showy();       // calls base2's function
  return 0;
}
OUTPUT:
10
20
```

Q. Explain the order of constructor and destructor execution in single and multilevel inheritance in C++. Support your answer with an example and output.

- In inheritance:
  - Constructors are executed in the order of derivation, from base to most-derived class.
  - Destructors are executed in reverse order, from most-derived to base class.
- This ensures:
  - All base class components are initialized before the derived class begins its own initialization.
  - Cleanup is done from the most-derived class backwards, ensuring proper destruction.

<u>Example: Multilevel Inheritance with Constructors and Destructors</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
public:
  base() { cout << "Constructing base\n"; }
  ~base() { cout << "Destructing base\n"; }
};

class derived1 : public base {
public:
  derived1() { cout << "Constructing derived1\n"; }
  ~derived1() { cout << "Destructing derived1\n"; }
};

class derived2: public derived1 {
public:
  derived2() { cout << "Constructing derived2\n"; }
  ~derived2() { cout << "Destructing derived2\n"; }
};

int main() {
  derived2 ob;
  // construct and destruct ob
  return 0;
}
```

OUTPUT:

```
Constructing base
Constructing derived1
Constructing derived2
Destructing derived2
Destructing derived1
Destructing base
```

Q. How are parameters passed to base-class constructors in C++ inheritance? Explain with an example and write the output.

- In C++, when a base class has a parameterized constructor, the derived class must pass arguments to it.

This is done using an expanded constructor initialization list in the derived class:

SYNTAX:

```
derived(args) : base(args) { ... }
```

- The derived class's constructor must:
  - Declare all parameters needed by both base and derived class.
  - Pass required arguments to the base constructor using the initialization list.
- The arguments passed can be expressions, variables, or function calls valid at the point of initialization.

<u>Example: Passing Parameters to Base-Class Constructor</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
protected:
  int i;
public:
  base(int x) { i = x; cout << "Constructing base\n"; }
  ~base() { cout << "Destructing base\n"; }
};

class derived : public base {
  int j;
public:
  // derived uses x; y is passed to base
  derived(int x, int y) : base(y) {
    j = x;
    cout << "Constructing derived\n";
  }

  ~derived() { cout << "Destructing derived\n"; }

  void show() { cout << i << " " << j << "\n"; }
};

int main() {
  derived ob(3, 4);
  ob.show();  // displays 4 3
  return 0;
}
```

OUTPUT:

```
Constructing base
Constructing derived
4 3
Destructing derived
Destructing base
```

Q. How can access to inherited members be restored in C++ when a base class is privately inherited? Explain with syntax and rules.

- When a class is inherited as private, all its public and protected members become private in the derived class.
- To restore original access (e.g., make a member public again), C++ provides two mechanisms:
    1. `using` declaration (preferred method in modern C++)
    2. Access declaration (deprecated but still supported)

<u>Access Declaration Syntax (Deprecated):</u>

```
base-class::member;
```

- This declaration must be placed under the appropriate access section (`public`, `protected`) of the derived class.
- No type specifier is used.

<u>Rules:</u>

- Only public and protected members can have their access restored.
- Private members of base class cannot be made public or protected.
- This mechanism does not change access control inside the derived class—it only affects the access level of inherited members outside the class.

<u>Example (Conceptual Fragment):</u>

```
class base {
public:
  int j; // public in base
};

// Inherit base as private
class derived : private base {
public:
  base::j;  // Restores j to public status in derived
};
```

- Access declarations allow selected inherited members to be re-exposed to the outside world under original access, despite private inheritance.

- They are deprecated in modern C++, and the `using` directive should be preferred for new code.

Q. What are virtual members in C++? Explain their role in achieving polymorphism.

- Polymorphism is the ability of different class objects to respond to the same function call (message) in different ways.
- To achieve polymorphism, it must be possible to refer to objects without knowing their specific class type.
- This requires the use of a single pointer variable that can refer to objects of different classes.
- A pointer to the base class can be used to refer to objects of all its derived classes.
- This is possible only when the function in the base class is declared virtual, allowing the correct function to be resolved at runtime based on the actual object the pointer refers to.

Q. What is the purpose of virtual base classes in C++? Explain with an example and provide the output.

- In multiple inheritance, if two or more classes inherit from the same base class, and a further class inherits from those classes, it can lead to multiple copies of the base class within the final derived class.
- This causes ambiguity when accessing members of the base class.
- To prevent this, the base class is inherited as a virtual base class by using the keyword `virtual`.
- If virtual base classes are used, then only one base class is present in the object. Otherwise, multiple copies will be found.

<u>Example: Using Virtual Base Classes</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
public:
  int i;
};

// derived1 inherits base virtually
class derived1 : virtual public base {
public:
  int j;
};

// derived2 also inherits base virtually
class derived2 : virtual public base {
public:
  int k;
```

```
};

// derived3 inherits both derived1 and derived2
class derived3 : public derived1, public derived2 {
public:
  int sum;
};

int main() {
  derived3 ob;
  ob.i = 10;     // only one shared copy of base
  ob.j = 20;
  ob.k = 30;
  ob.sum = ob.i + ob.j + ob.k;

  cout << ob.i << " ";
  cout << ob.j << " " << ob.k << " ";
  cout << ob.sum;
  return 0;
}

OUTPUT:

10 20 30 60
```

Q. What is a virtual function in C++? Explain its behavior with an example and write the output.

- A virtual function is a member function declared in a base class using the keyword `virtual`.
- It is redefined in the derived class to implement behavior specific to that class.
- A virtual function implements the concept of "one interface, multiple methods".
- When a base-class pointer points to a derived object containing a virtual function:
  - C++ determines which version of the function to call at run time, based on the actual object.
- Run-time polymorphism is achieved only when accessed through a base-class pointer or reference.
- Redefinition of a virtual function must exactly match its prototype; otherwise, it will be treated as an overloaded function, and polymorphism will not work.
- Virtual functions must be nonstatic, cannot be friends, and constructors cannot be virtual. Destructors can be virtual.

<u>Example: Virtual Function Implementation</u>

```
INPUT CODE:
#include <iostream>
using namespace std;
```

```
class base {
public:
  virtual void vfunc() {
    cout << "This is base's vfunc().\n";
  }
};

class derived1 : public base {
public:
  void vfunc() {
    cout << "This is derived1's vfunc().\n";
  }
};

class derived2 : public base {
public:
  void vfunc() {
    cout << "This is derived2's vfunc().\n";
  }
};

int main() {
  base *p, b;
  derived1 d1;
  derived2 d2;

  p = &b;
  p->vfunc(); // access base's vfunc()

  p = &d1;
  p->vfunc(); // access derived1's vfunc()

  p = &d2;
  p->vfunc(); // access derived2's vfunc()

  return 0;
}

OUTPUT:

This is base's vfunc().
This is derived1's vfunc().
This is derived2's vfunc().
```

Q. How can a virtual function be called through a base class reference in C++? Explain with an example and write the output.

- A base-class reference can refer to:
  - An object of the base class.

    - An object of any class derived from the base.
- When the base class contains a virtual function, and the function is accessed via a base-class reference, run-time polymorphism is achieved.
- The actual type of the object referred to by the reference determines which version of the virtual function gets executed.

<u>Example: Calling a Virtual Function Through a Base-Class Reference</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
public:
  virtual void vfunc() {
    cout << "This is base's vfunc().\n";
  }
};

class derived1 : public base {
public:
  void vfunc() {
    cout << "This is derived1's vfunc().\n";
  }
};

class derived2 : public base {
public:
  void vfunc() {
    cout << "This is derived2's vfunc().\n";
  }
};

// Function taking base class reference
void f(base &r) {
  r.vfunc();
}

int main() {
  base b;
  derived1 d1;
  derived2 d2;

  f(b);   // passes base object
  f(d1);  // passes derived1 object
  f(d2);  // passes derived2 object

  return 0;
}
```

```
OUTPUT:
This is base's vfunc().
This is derived1's vfunc().
This is derived2's vfunc().
```

Q. What are the rules for virtual functions in C++?

- A virtual function must be a member of some class.
- A virtual function cannot be static.
- Virtual functions are accessed using object pointers, typically pointers to the base class.
- A virtual function can be a friend of another class.
- A virtual function in a base class must be defined, even if it is not used.
- The prototypes of the base class version and all derived class versions must be identical.
- Virtual constructors are not allowed, but virtual destructors are allowed.
- A base class pointer can point to any derived class object, but a derived class pointer cannot point to a base class object.
- When a base pointer points to a derived object, incrementing or decrementing the pointer does not move it to the next derived object.
- If a virtual function is not overridden in the derived class, the base class version is invoked.

Q. State and explain the key properties of virtual functions in C++.

1. The Virtual Attribute Is Inherited

    - When a class inherits a virtual function, its virtual nature is automatically preserved.
    - If this derived class is used as a base class for another class, the function remains virtual and can still be overridden.
    - This applies across multiple levels of inheritance.
    - A virtual function stays virtual no matter how many times it is inherited.

2. Virtual Functions Are Hierarchical

    - Once a function is declared as virtual in a base class:
        - It may be overridden by any derived class.
        - It is not required to be overridden.
    - If a derived class does not override the virtual function:
        - The base class version is used when the function is called on an object of the derived class.

Q. What is a pure virtual function in C++? Write a simple example.

- A pure virtual function has no definition in the base class.
- It serves as a placeholder to be redefined by derived classes.
- It is declared by assigning `= 0` in its declaration:
- Syntax:

```
class classname {
public:
  virtual void virtualfunctionname() = 0;  // Pure virtual
function
};
```

- A class containing at least one pure virtual function is called an abstract class.
- Any derived class must override the pure virtual function.
- If the derived class does not override it, it also becomes abstract and cannot be instantiated.
- Pure virtual functions are used to enforce overriding.
- They define a contract that derived classes must implement.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class base {
public:
  virtual void show() = 0; // pure virtual function
};

class derived : public base {
public:
  void show() {
    cout << "Derived class show().\n";
  }
};

int main() {
  derived d;
  d.show();
  return 0;
}
OUTPUT:
Derived class show().
```

Q. What is an abstract class in C++? Explain with rules and an example.

- An abstract class is a class intended for use as a base class.
- It contains at least one pure virtual function, declared using `= 0`.
  Example:

```
class AB {
public:
  virtual void f() = 0;
};
```

- A class that contains at least one pure virtual function is incomplete, and no objects of such a class can be created.
- Pure virtual functions are declared without a body:

```
virtual void func() = 0;
```

- A function declaration cannot have both a pure specifier and a function body. Example (Invalid):

```
class A {
  virtual void g() { } = 0;  // Error
};
```

- Abstract classes cannot be used as:
  - A function return type
  - A function parameter type
  - The type of an explicit conversion
- You can:
  - Declare pointers and references to an abstract class.
  - Use abstract class pointers/references to support run-time polymorphism through base-class pointers or references.

<u>Example:</u>

```
class A {
  virtual void f() = 0;   // Pure virtual function
};

class B : public A {
  void f() { }            // Overrides pure virtual function
};

int main() {
  // A a;        // Error: Cannot instantiate abstract class
  A* pa;         // OK: Pointer to abstract class
  B b;           // OK: Derived class provides definition
}
```

Q. How are virtual functions used in C++? Explain the concept with its purpose.

- A base class can define the interface to a general class using virtual functions.
- Each derived class then provides its own implementation of the virtual function, based on its specific data and needs.
- This enables the "one interface, multiple methods" approach, using:
  - Virtual functions
  - Abstract classes
  - Run-time polymorphism
- All common features and interfaces are defined in the base class.

- For actions that depend on the specific derived class, use virtual functions so that derived classes can implement them accordingly.
- Abstract classes are used when certain functions must be implemented by derived classes.
- A key use of virtual functions and abstract classes is in creating class libraries, where:
    - The base class interface is fixed.
    - Other programmers can extend or adapt the base class behavior in their own derived classes as needed.

Q. What is polymorphism in C++? Explain its types.

- **Polymorphism** is the phenomenon where the **same message sent to two different objects** produces **two different sets of actions**.
- Polymorphism is broadly divided into **two types**:

  • **Static Polymorphism**
  – Achieved through **function overloading**.
  – Also known as **compile-time polymorphism**.

  • **Dynamic Polymorphism**
  – Achieved through **late binding** using **virtual functions**.
  – Also known as **run-time polymorphism**.

Q. What is static polymorphism in C++? Explain with characteristics.

- Static polymorphism refers to an entity existing in different physical forms simultaneously.
- It involves binding functions based on the number, type, and sequence of arguments.
- The function parameters are specified in the function declaration, so the correct function is bound to a call at compile time.
- This is called early binding, static binding, or static linking.
- The term early binding means the calls to functions are already resolved before program execution.
- The compiler determines the appropriate function version using:
    - Number of arguments
    - Type of arguments
    - Sequence of arguments

<u>Example Function Declarations:</u>

```
void add(int, int);

void add(float, float);
```

- When `add()` is called, the passed parameters decide which version will be executed.
- This resolution happens at compile time, making it static polymorphism.

Q. What is dynamic polymorphism in C++? Explain with characteristics.

- Dynamic polymorphism refers to an entity changing its form depending on the circumstances.
- A function exhibits dynamic polymorphism when:
  - It exists in multiple forms.
  - The correct version is selected at run-time, not compile time.
- This process is called late binding.
- In late binding, the function call is resolved during execution, allowing:
  - Greater flexibility.
  - The appropriate method to be invoked based on the actual object at run-time.
- Dynamic polymorphism is implemented using virtual functions.

Q. Differentiate between static and dynamic polymorphism in C++.

Feature	Static Polymorphism	Dynamic Polymorphism
Binding Time	Compile-time (early binding)	Run-time (late binding)
Efficiency	More efficient	More flexible
Function Resolution	Resolved at compile time	Resolved during program execution
Runtime Overhead	No runtime search required	Additional runtime search required
Code Modification	Requires major changes for requirement updates	Allows flexibility with minimal code changes

Usage Example	Function overloading	Virtual functions

Q. Explain pointers to objects in C++ with the help of an example.

- Like any other data type, pointers can also be used with class objects.
- A pointer to an object is declared using the class name followed by `*`.
- The pointer can be used to access members of the class using either:
  - Arrow operator (`->`)
  - Dereference operator (`*`) and dot operator (`.`)

<u>Example Code:</u>

```
INPUT CODE:
class alpha {
  int x;
public:
  void set(int i) {
    x = i;
  }

  void show_alpha() {
    cout << x << "\n";
  }
};

alpha obj;
alpha *ptr;

ptr = &obj;              // ptr now points to obj
ptr->show_alpha();       // accessing member using arrow
operator
ptr->set(10);            // setting value using arrow operator
(*ptr).show_alpha();     // accessing member using dereference
and dot operator
OUTPUT:
0     // initial value (default garbage or uninitialized)
10    // value after calling set(10)
```

<u>Explanation:</u>

- `alpha *ptr;` → declares a pointer `ptr` to an object of class `alpha`.
- `ptr = &obj;` → assigns the address of `obj` to `ptr`.
- `ptr->show_alpha();` → calls `show_alpha()` using pointer.
- `ptr->set(10);` → sets `x = 10` in `obj` through the pointer.

- `(*ptr).show_alpha();` → dereferences the pointer and accesses `show_alpha()` using dot operator.

Q. What is the `this` pointer in C++? Explain with an example.

- The `this` pointer is a **special pointer** available **only inside non-static member functions** of a class, struct, or union.
- It **points to the calling object** — the object for which the member function is currently being executed.
- **Static member functions do not have access to `this`**, as they are not called on an instance.
- For example, when `A.max()` is called, `this` points to the **address of object A**.

<u>Usage Example:</u>

```
class xyz {
  int x;
  int y;
public:
  void set(int a, int b) {
    this->x = a;
    this->y = b;
  }
};
```

- `this->x = a;` assigns the value `a` to the member variable `x` of the **calling object**.

<u>Application of `this` pointer:</u>

- It is commonly used when a function needs to **return the current object**.
- `return *this;` returns the **object itself**.

<u>Example:</u>

```
class Person {
  int age;
public:
  Person(int a) { age = a; }

  Person &greater(Person &x) {
    if (x.age > age)
      return x;
    else
      return *this;
  }
};
```

- `greater()` takes another `Person` object and compares `age`.

- If `x.age` is greater, it returns `x`.
- Otherwise, it returns `*this` — the current object.

Q. Explain pointers to derived classes in C++ with an example.

- In C++, a pointer to a derived class is type-compatible with a pointer to its base class.
- This allows a base class pointer to point to objects of any derived class.
- This is useful in achieving polymorphism, especially when using virtual functions.

<u>Example:</u>

```
INPUT CODE:
class CPolygon {
protected:
  int width, height;
public:
  void set_values(int a, int b) {
    width = a;
    height = b;
  }
};

class CRectangle : public CPolygon {
public:
  int area() {
    return (width * height);
  }
};

class CTriangle : public CPolygon {
public:
  int area() {
    return (width * height / 2);
  }
};

int main() {
  CRectangle rect;
  CTriangle trgl;
  CPolygon *ppoly1 = &rect;
  CPolygon *ppoly2 = &trgl;

  ppoly1->set_values(4, 5);
  ppoly2->set_values(4, 5);

  cout << rect.area() << endl;
  cout << trgl.area() << endl;
}
OUTPUT:
```

```
20
10
```

Explanation:

- `ppoly1` and `ppoly2` are base class (`CPolygon`) pointers.
- They point to derived class objects: `rect` and `trgl`.
- They are used to call the `set_values()` function defined in the base class.
- The actual call to `area()` is made through the derived class objects directly.

IMPORTANCE OF DEFAULT FUNCTION ARGUMENTS IN C++ PROGRAMMING

Definition:

- Default arguments are values provided in a function declaration.
- These values are automatically used by the compiler if no argument is passed by the caller for those parameters.

Importance:

- Flexibility in Function Calls:
  - Enables calling a function with varying numbers of arguments.
  - Allows writing one function for multiple use cases.
- Code Simplification:
  - Reduces the need for multiple overloaded functions.
  - Simplifies code by handling optional parameters directly.
- Better Readability and Maintainability:
  - Makes the intention of default behavior clear.
  - Easy to update default behavior in one place.
- Improved Reusability:
  - Same function can be reused with different sets of parameters.

Syntax:

```
ReturnType FunctionName(Parameter1 = default1, Parameter2 =
default2, ...);
```

- Default values are specified in the function **declaration**, not in the definition.

Example:

```
#include <iostream>
using namespace std;

void showMessage(string msg = "Hello", int times = 1) {
  for(int i = 0; i < times; i++)
```

```
    cout << msg << endl;
}

int main() {
  showMessage();                // Uses both defaults
  showMessage("Welcome");       // Uses default for times
  showMessage("Bye", 3);        // No default used
  return 0;
}
```

OUTPUT:

```
Hello
Welcome
Bye
Bye
Bye
```

Rules:

- All default arguments must be **trailing** (i.e., appear at the end of the parameter list).
- Cannot skip arguments in the middle of the list.
- Once a default is provided for a parameter, all parameters to its right must also have defaults.

OPERATOR OVERLOADING IN C++

Definition:

- Operator overloading allows you to redefine the way operators work for user-defined types (like objects).
- It provides polymorphism in C++.
- You can overload most operators (e.g., +, -, *, =, <<, >>), but some operators cannot be overloaded (e.g., `::`, `.`).

Importance:

- Improves readability and intuitiveness of code.
- Enables operations on objects just like primitive types.

Syntax:

```
ReturnType operator OperatorSymbol (ArgumentList);
```

- Defined as a member function or friend function.

Example Program:

This program adds two complex numbers using overloaded + operator.

```
#include <iostream>
using namespace std;

class Complex {
  float real, imag;

public:
  // Constructor
  Complex(float r = 0, float i = 0) {
    real = r;
    imag = i;
  }

  // Overload + operator
  Complex operator + (Complex obj) {
    Complex temp;
    temp.real = real + obj.real;
    temp.imag = imag + obj.imag;
    return temp;
  }

  // Function to display complex number
  void display() {
    cout << real << " + " << imag << "i" << endl;
  }
};

int main() {
  Complex c1(2.5, 3.5), c2(1.6, 2.4), result;

  result = c1 + c2;  // Calls overloaded + operator

  cout << "Sum = ";
  result.display();  // Output: 4.1 + 5.9i

  return 0;
}
```

Explanation:

- `operator+()` is overloaded to add two `Complex` objects.
- The operator works like `c1 + c2`, just like for built-in types.

DISTINCTION BETWEEN STATIC AND DYNAMIC POLYMORPHISM IN C++

POLYMORPHISM

- Means "one interface, multiple methods".
- Allows the same function or operator to behave differently based on context.

STATIC POLYMORPHISM (Compile-Time Polymorphism)

Definition:

- Polymorphism resolved at **compile time**.
- Achieved through **function overloading** and **operator overloading**.

Features:

- Binding occurs during compilation.
- Faster execution.
- Less flexible (cannot change behavior at runtime).

Example: Function Overloading

```
#include <iostream>
using namespace std;

class Print {
public:
  void show(int i) {
    cout << "Integer: " << i << endl;
  }

  void show(double d) {
    cout << "Double: " << d << endl;
  }
};

int main() {
  Print obj;
  obj.show(5);       // Integer version
  obj.show(3.14);    // Double version
  return 0;
}
```

Explanation:

- Two versions of `show()` based on parameter type.
- Compiler selects the correct version at compile time.

DYNAMIC POLYMORPHISM (Run-Time Polymorphism)

- Polymorphism resolved at **runtime**.
- Achieved through **virtual functions** and **inheritance**.

Features:

- Binding occurs during execution.
- Slightly slower due to virtual table lookup.
- More flexible and extensible.

Example: Virtual Function

```
#include <iostream>
using namespace std;

class Base {
public:
  virtual void display() {
    cout << "Display of Base class" << endl;
  }
};

class Derived : public Base {
public:
  void display() {
    cout << "Display of Derived class" << endl;
  }
};

int main() {
  Base *ptr;
  Derived obj;
  ptr = &obj;

  ptr->display();  // Calls Derived version due to virtual
function
  return 0;
}
OUTPUT:
Display of Derived class
```

Explanation:

- `display()` is virtual in base class.
- `ptr` points to a `Derived` object.
- Function call resolved at **runtime**.

COMPARISON TABLE

Aspect	Static Polymorphism	Dynamic Polymorphism

Binding Time	Compile-time	Run-time
Techniques Used	Function Overloading, Operator Overloading	Virtual Functions, Inheritance
Speed	Faster	Slower
Flexibility	Less Flexible	More Flexible
Example	`show(int)`, `show(double)`	Overridden `display()` in base/derived
Keywords Used	None specific	`virtual`

CODE FOR Overloading ++ and -- Operators Using Friend Functions

Program:

```
#include <iostream>
using namespace std;

class Number {
  int x;

public:
  Number() {
    x = 0;
  }

  Number(int a) {
    x = a;
  }

  // Friend functions to overload ++ and --
  friend Number operator++(Number &obj, int);  // postfix ++
  friend Number operator--(Number &obj, int);  // postfix --

  void show() {
    cout << "x = " << x << endl;
```

```
  }
};

// Postfix ++ (dummy int means it's postfix)
Number operator++(Number &obj, int) {
  Number temp = obj;
  obj.x = obj.x + 1;
  return temp;
}

// Postfix --
Number operator--(Number &obj, int) {
  Number temp = obj;
  obj.x = obj.x - 1;
  return temp;
}

int main() {
  Number n(5);

  cout << "Start: ";
  n.show();

  Number a = n++;  // n becomes 6
  cout << "After n++: ";
  n.show();
  cout << "Returned from n++: ";
  a.show();

  Number b = n--;  // n becomes 5
  cout << "After n--: ";
  n.show();
  cout << "Returned from n--: ";
  b.show();

  return 0;
}
```

OUTPUT:

```
Start: x = 5
After n++: x = 6
Returned from n++: x = 5
After n--: x = 5
Returned from n--: x = 6
```